AIORI-3 HACKATHON · CATEGORY: CYBER SECURITY · PS-005



Explainable BGP Origin and Subprefix Anomaly Monitor

Time-aligned RPKI validation, operator inventory checks and evidence-based alerts, evaluated honestly.

TEAM

ThirdEye

Project report · October 2026

Repository: github.com/aiori-hackathon/A3-PS005-TC512

Reference: RFC 6811, BGP Prefix Origin Validation

2. Table of Contents

3. Problem Description . . . 3

Key issues . . . 3

What the monitor must therefore do . . . 3

4. Solution Proposed . . . 4

Pipeline stages . . . 4

Detection layer 1: rule-based baseline . . . 4

Detection layer 2: ML detector . . . 4

Explainable alerts . . . 4

Labelled synthetic scenarios . . . 5

5. Optimization Proposed by the Team . . . 6

Before and after . . . 6

Measured effect on the held-out test slice . . . 6

Honesty checks . . . 8

6. Solution Architecture and Design . . . 9

Design principles . . . 9

Detection decision flow (rule baseline) . . . 10

Detection loop (pseudocode) . . . 10

Technology stack . . . 11

7. Timeline of Delivery . . . 12

8. References . . . 13

Standards and specifications . . . 13

Data formats supported by the loaders . . . 13

Tools and libraries . . . 13

9. Conclusion . . . 14

Why it matters . . . 14

Limitations . . . 14

Next steps . . . 14

3. Problem Description

The Border Gateway Protocol (BGP) glues the Internet together, yet it was designed without origin verification. Any network can announce prefixes it does not own, by misconfiguration or on purpose, and neighbouring networks will usually accept the announcement and send traffic towards it. Resource Public Key Infrastructure (RPKI) Route Origin Authorizations (ROAs) improve this by signing a prefix-to-ASN mapping with a maximum prefix length, and routers can apply RFC 6811 route origin validation (ROV). Origin validation alone, however, leaves important gaps that an operator has to monitor.

Key issues

- **Trust-based routing.** BGP accepts announcements without checking who owns the prefix, so hijacks and leaks redirect traffic.
- **Subprefix (more-specific) hijacks.** Routers prefer the longest matching prefix, so a /24 announced inside a legitimate /22 wins traffic. A loose ROA maxLength can even authorise such more-specifics.
- **Forged-origin attacks.** An attacker announces the more-specific using the *legitimate* origin ASN. ROV says Valid and the origin is in the operator inventory, so origin-only checks are blind to it.
- **Legitimate change looks like an attack.** Anycast, multi-homing and DDoS-scrubbing providers create multiple legitimate origins, and policy changes make ROAs or the inventory lag behind announcements. A naive detector drowns the operator in false alarms.
- **Stale or misaligned reference data.** Judging an announcement against today's ROAs instead of the ROAs valid at that moment gives wrong verdicts.
- **Alerts without evidence.** If an alert does not say which ROA, which inventory entry and which rule were involved, an analyst cannot triage it.
- **No honest measurement.** Detectors are rarely tested on labelled events with legitimate hard cases, so their false-positive cost is unknown.

What the monitor must therefore do

Need	How the problem statement frames it
Detect unexpected origins and subprefix / more-specific anomalies	Match announcements against time-aligned ROA tables and an operator prefix inventory
Explain every detection	Alerts show clear evidence of why an announcement was flagged
Measure honestly	Precision, recall, alert latency and false positives, including legitimate multi-origin and policy-change scenarios
Stay within scope	Rule-based baseline first, one ML model afterwards. **No** automatic route blocking, blackholing or full AS_PATH analysis

4. Solution Proposed

ThirdEye is a BGP update monitor that replays a routing feed, validates every announcement against the ROA snapshot that was current at that time and against the operator's own prefix inventory, and raises alerts that carry their own evidence. It runs two detection layers on the same stream: a transparent **rule-based baseline** built on RFC 6811, and **one machine-learning model** that was added only after the baseline had been evaluated.

Pipeline stages

Stage	What it does
BGP replay	Streams normalized updates (`timestamp, prefix, origin_asn, peer, action`) in strict time order. Loaders exist for normalized CSV and raw collector dumps (RIPE RIS / RouteViews format).
Time-aligned ROA store	For an update at time *t*, uses the latest snapshot at or before *t*. If none exists the outcome is `NoSnapshot`; it never falls back to a later snapshot.
Prefix inventory	Operator-owned prefixes with their allowed origin ASNs. Exact and longest-prefix-match lookup for IPv4 and IPv6.
RFC 6811 validator	Returns Valid, Invalid, NotFound or NoSnapshot, plus the covering ROAs and the reason for invalidity (origin mismatch, length above maxLength, AS0 ROA).
Rule-based baseline	Strict rule precedence so each update yields at most one alert, with deduplication that never suppresses a new origin.
ML detector	Gradient-boosted classifier on ten past-only features, trained with a chronological split, threshold chosen on validation data only.
Evidence builder	Turns every detection into a structured JSON alert with covering ROAs, snapshot time, ROV state, inventory match, peer and a plain-English reason.
Evaluator	One-to-one matching of alerts to labelled events; precision, recall, F1, per-scenario recall, false positives per benign scenario, bootstrap confidence intervals, multiple seeds.

Detection layer 1: rule-based baseline

Alert type	Fires when	Severity
ORIGIN_HIJACK	Exact inventory prefix announced by an origin outside the allowed set, ROV Invalid or NotFound	High
SUBPREFIX_HIJACK	More-specific of an inventory or ROA prefix announced by an unauthorised origin, ROV Invalid or NotFound	High
INVENTORY_MISMATCH	ROV Valid but the origin is not in the inventory (configuration drift, not an attack)	Low
ROA_INVALID	Origin is in the inventory but ROV is Invalid (the ROA lags a policy change)	Medium
ROA_MISSING	Inventory-owned prefix with no covering ROA	Low
THIRD_PARTY_INVALID	RPKI-invalid route *outside* the inventory; reported separately and excluded from operator precision	Low

Detection layer 2: ML detector

The rules cannot see a forged-origin more-specific, because both ROV and the inventory say the announcement is fine. The ML model is trained to separate such attacks from legitimate traffic-engineering more-specifics using feed metadata only: RFC 6811 state, origin-in-inventory, prefix-length delta, origin new for the prefix, **peer new for this prefix and origin**, more-specific first seen, prefix age (capped at 6 hours), origin rarity, update rate over 30 minutes and hour of day. All features are computed from past data only. No raw prefix, raw ASN or event identifier is used as a feature, and labels come from the ground-truth file, never from rule outputs.

Explainable alerts

Every alert from either detector is one JSON line with a deterministic ID. Rule alerts state which rule fired and why earlier rules in the precedence did not. ML alerts add the score, the threshold and the top three contributing features, computed as a per-feature contribution against a training-set background and labelled as a non-causal model attribution. Analysts can inspect alerts with `python -m src.cli explain --alert-id X` and filter them with `list-alerts --type T --severity S`.


```
{ "alert_id": "alt-ml-7a7b0b3b", "type": "ML_ANOMALY", "detector": "ml",
  "prefix": "198.19.35.0/24", "origin_asn": 64510, "score": 0.1977, "threshold": 0.05,
  "evidence": {
    "covering_roas": [{"prefix": "198.18.0.0/15", "asn": 64510, "maxLength": 24}],
    "rfc6811_state": "Valid",
    "inventory_match": {"prefix": "198.18.0.0/15", "owner": "Enterprise Campus",
                        "allowed_origins": [64510]},
    "peer": "206.108.34.1",
    "top_contributing_features": [ {"feature": "origin_rarity", "value": 0.0004},
                                   {"feature": "peer_new_for_prefix_origin", "value": 1.0}, ... ] } }
```


Example of an alert the rules cannot raise: ROV is Valid and the origin is authorised, yet the announcement arrives via a peer that has never carried this origin for this prefix.

Labelled synthetic scenarios

A seeded, YAML-configurable generator injects labelled events into the replay and writes a ground-truth file with scenario, difficulty and expected alert type. Legitimate hard cases are included on purpose, so false positives can be measured rather than assumed away.

Class	Scenario	What it models
Anomaly	exact_origin_hijack (easy)	Foreign ASN announces an inventory prefix
Anomaly	subprefix_hijack (easy)	Foreign ASN announces a more-specific inside an inventory prefix
Anomaly	hijack_after_roa_change (hard)	Hijack minutes after a ROA snapshot change; tests time alignment
Anomaly	loose_maxlength_hijack (hard)	Forged-origin more-specific: legitimate ASN, within maxLength, ROV Valid
Benign	multi_origin_anycast	Several legitimate, authorised origins
Benign	multi_origin_ddos_scrubbing	Provider authorised by ROA but missing from the inventory
Benign	policy_change_roa_first	ROA published before the new origin announces
Benign	policy_change_roa_lag_inventory_updated / _stale	New origin announces before the ROA catches up; inventory current or stale
Benign	te_more_specific	Legitimate traffic-engineering more-specifics within maxLength

The rule baseline was evaluated on 3,164 updates and 92 labelled events (40 anomalies, 52 benign). The ML experiments use a ten-times larger dataset with 1,020 labelled events, its own inventory and ROA snapshots, and a chronological 60 / 20 / 20 split in which every slice contains every scenario. This is a scale change, not a scenario change.

5. Optimization Proposed by the Team

The team treated evaluation itself as something to optimise. Each improvement below was found by auditing the previous result, and the earlier results were preserved so the progression can be inspected. All figures are produced by code and stored in the repository's `reports/` folder.

Before and after

Optimization	Before	After
Add ML after the baseline	Forged-origin recall 0%; overall test recall 0.72	Forged-origin recall about 81%; overall test recall 0.92
Separate INVENTORY_MISMATCH	ROV-valid announcement from an unregistered origin labelled as a hijack	Reported as a low-severity configuration-drift alert; hijack types only when ROV is Invalid or NotFound
Separate THIRD_PARTY_INVALID	RPKI-invalid noise outside the inventory counted against operator precision	19 such alerts reported in their own noise table
Strict one-to-one benign matching	Greedy window let one event absorb its neighbours' alerts (8 alerts, 6 events flagged)	Every event maps to exactly one alert; regression test added
Difficulty aggregation fix	Easy 32/32 and hard 8/16, which sums to 48 events for 40 anomalies	Easy 24/24 and hard 8/16; a test asserts the totals
Generator v1 audit	Edits after the first run had silently removed background noise (126 to 58 to 0 alerts)	Disclosed; v1 counts preserved; forged-origin scenario made genuinely hard; third-party noise added
ML leakage removal	F1 of 1.00 and a shuffled-label AUC of 0.75 then 0.31; features separated classes almost perfectly	Peer randomisation, legitimate-origin carrier traffic, capped prefix age; F1 0.91 ± 0.01 and shuffled AUC 0.55 ± 0.20, both disclosed
Dataset separation	ML changes had modified the shared inventory and ROA snapshots	ML has its own inputs; the baseline reproduces 0.6154 / 0.8000 / 0.6957 exactly
Reproducible reports	Random alert IDs and timings made every re-run dirty the reports	Deterministic IDs, timing in a separate file, idempotent runs

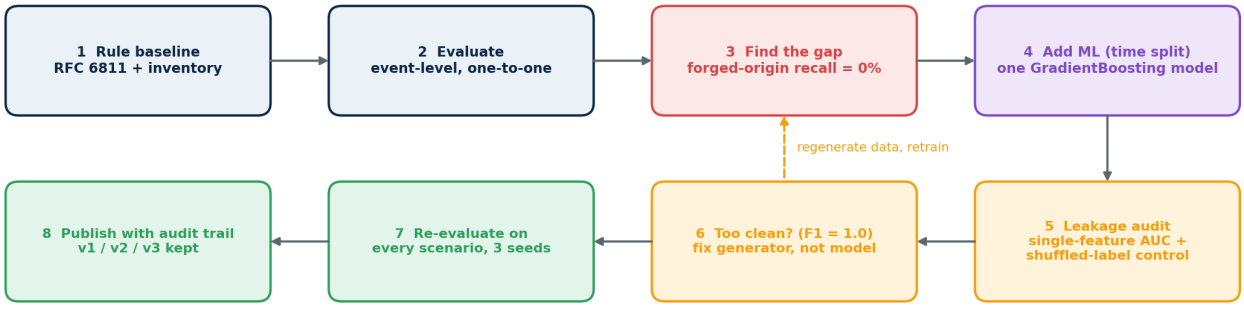


Figure 1. The optimization loop: baseline, evaluate, find the gap, add ML, audit for leakage, fix the data, re-evaluate.

Measured effect on the held-out test slice

Numbers are the mean of three generator seeds (777, 888, 999) on the chronological test slice, using event-level one-to-one matching.

System	Event precision	Event recall	Event F1	Latency per update
Rules (primary: all alert types)	0.5185	0.7242	0.6043	0.0145 ms
Rules (secondary: high and medium only)	0.6364	0.7242	0.6774	0.0145 ms
ML alone	**0.9044 ± 0.0325**	**0.9183 ± 0.0261**	**0.9104 ± 0.0087**	0.60 ms
Rules OR ML (primary)	0.5476	0.9183	0.6859	0.60 ms
Rules OR ML (secondary)	0.6476	0.9183	0.7593	0.60 ms

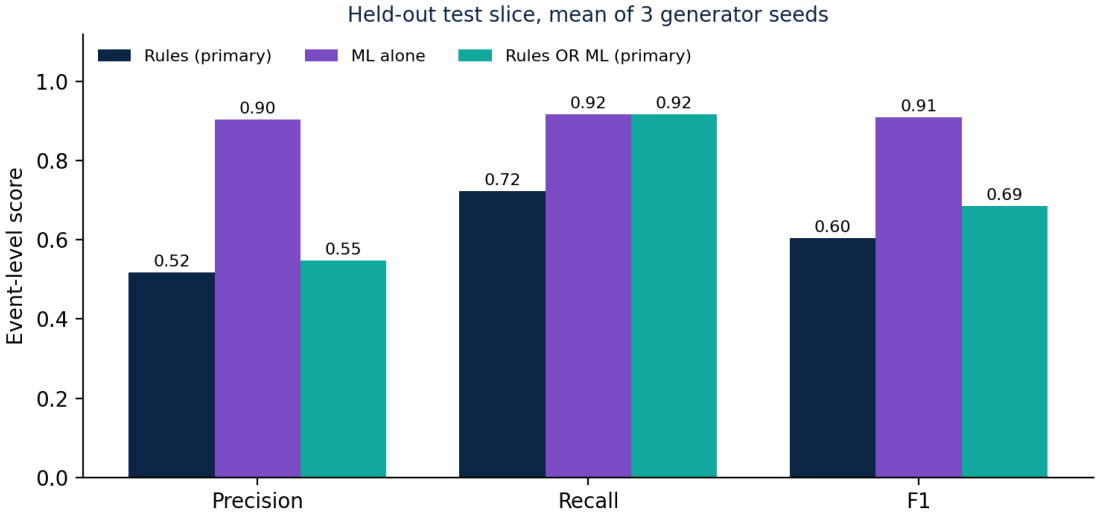


Figure 2. Rules versus ML versus the hybrid on the held-out test slice.

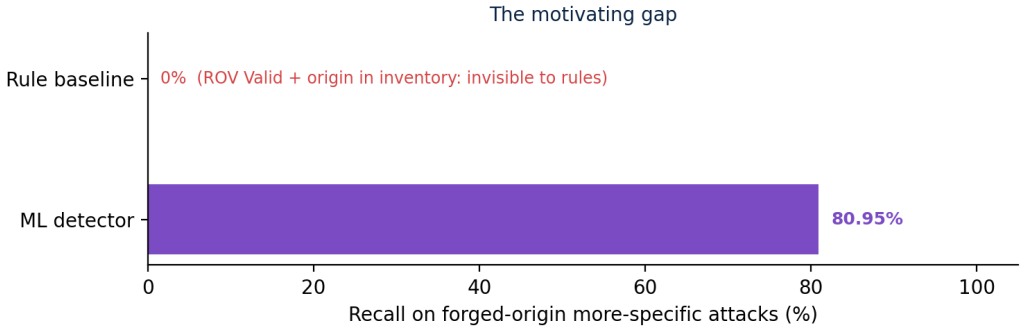


Figure 3. Forged-origin more-specific attacks are invisible to the rules and recovered in large part by ML.

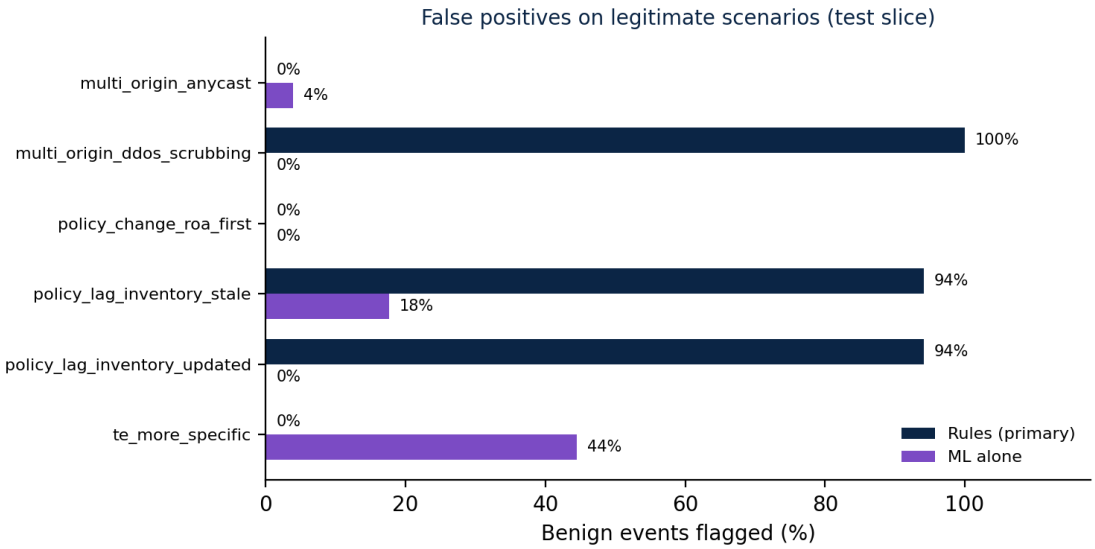


Figure 4. Share of legitimate events flagged, per scenario. ML removes most DDoS-scrubbing and policy-lag false positives but adds some on traffic-engineering more-specifics.

Trade-off. ML lifts forged-origin recall from 0% to about 81% and cuts false positives on DDoS-scrubbing and policy-lag events, but it flags 8 of 18 legitimate traffic-engineering events, and costs about 40 times the per-update processing time of the rules (0.60 ms against 0.0145 ms, measured end to end, one update at a time). Combining rules OR ML restores recall but inherits the rules' false positives, so ML alone is the best option on this data.

Honesty checks

- **Shuffled-label control** (20 permutations): test AUC mean 0.5522, std 0.1954 (min 0.157, max 0.844). The mean is near chance, but the spread is large because positives are clustered in a small slice. It shows no systematic leakage on average; it does not prove leakage is absent.
- **Residual separability:** the single feature `peer_new_for_prefix_origin` has an AUC of 0.893, above the team's goal of about 0.85. This is reported rather than hidden.
- **Seeds** 777, 888 and 999 measure variance across synthetic datasets from one generator. They are not external validity.

6. Solution Architecture and Design

The system is a streaming pipeline. Inputs (feed, ROA snapshots, inventory) are prepared and validated once, then shared by two detectors. Both detectors hand their findings to a single evidence builder, so every alert has the same structure whichever detector raised it. A separate evaluator compares alerts with the generator's ground-truth labels.

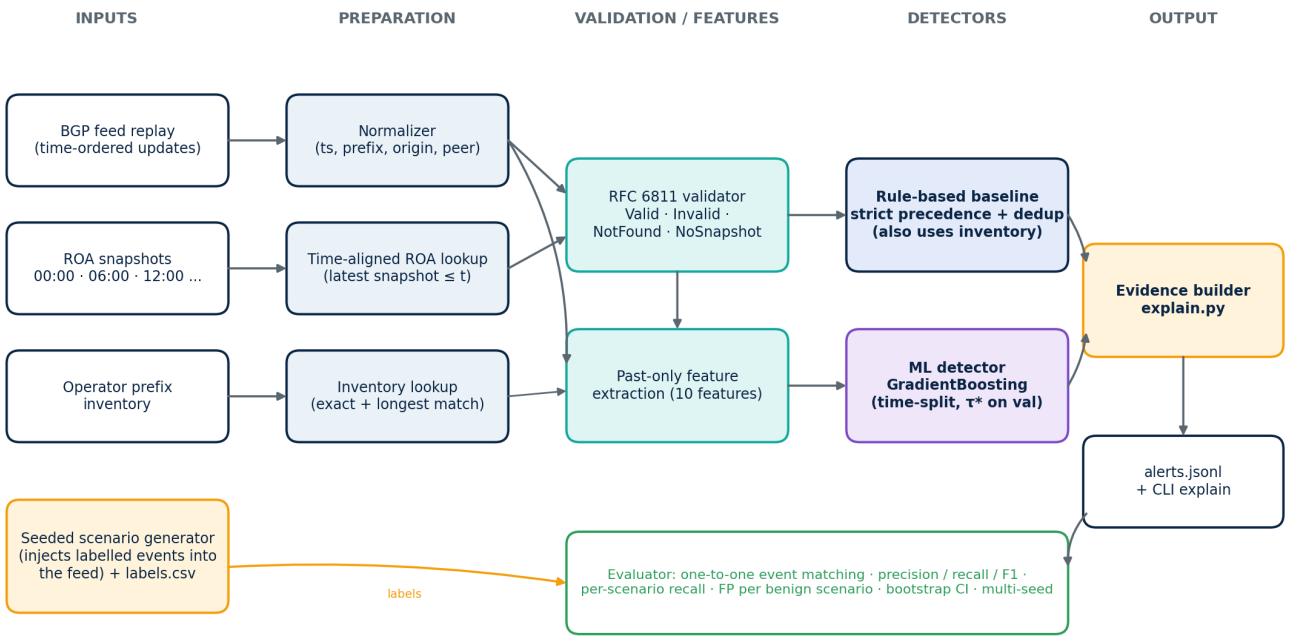


Figure 5. Solution architecture and data flow.

Design principles

- **Baseline first.** The ML model was added only after the rule baseline had been evaluated and its metrics saved.
- **No future leakage.** ROA lookup uses snapshots at or before the update time; ML features use past data only; data is split chronologically.
- **Explainability by construction.** Evidence is collected while the decision is made, not reconstructed afterwards.
- **Separation of concerns.** Replay, ROA store, validator, rules, features, model, explanation and evaluation are separate modules with their own tests.
- **Reproducibility.** Seeded generation, deterministic alert IDs, and a configuration file for every dataset.

Detection decision flow (rule baseline)

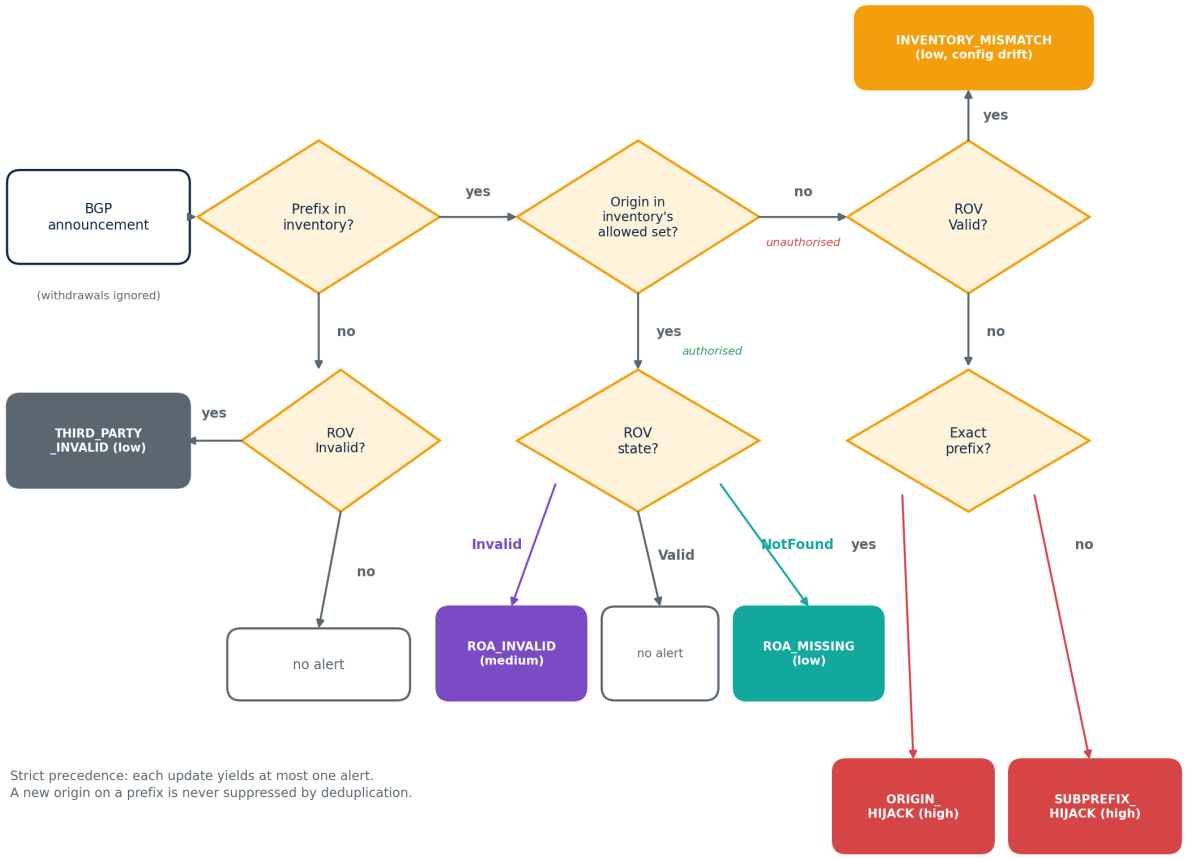


Figure 6. Strict rule precedence. Each announcement produces at most one alert.

Detection loop (pseudocode)

```
for update in replay(feed):                      # strict time order
    if update.action == WITHDRAW: continue        # only announcements are judged
    snap = roa_store.snapshot_at(update.time)     # latest snapshot <= time, else NoSnapshot
    rov  = validate_rfc6811(update.prefix, update.origin, snap)   # + covering ROAs, reason
    inv  = inventory.lookup_covering(update.prefix)

    # ---- layer 1: rule baseline ----
    if inv is None:
        if rov.state == INVALID: alert(THIRD_PARTY_INVALID, low)
    elif update.origin not in inv.allowed_origins:
        if rov.state == VALID:        alert(INVENTORY_MISMATCH, low)
        elif update.prefix == inv.prefix: alert(ORIGIN_HIJACK, high)
        else:                         alert(SUBPREFIX_HIJACK, high)
    elif rov.state == INVALID:        alert(ROA_INVALID, medium)   # ROA lags policy change
    elif rov.state == NOT_FOUND:      alert(ROA_MISSING, low)
    # (deduplicate on (prefix, origin) inside a window; a new origin is never suppressed)

    # ---- layer 2: ML detector ----
    x = features(update, rov, inv, history)       # past-only rolling state
    score = model.predict_proba(x)
    if score >= TAU_STAR:                         # TAU_STAR chosen on validation only
        alert(ML_ANOMALY, score, top3_contributions(x))

    history.update(update)                        # state is advanced after the decision
```

Technology stack

Layer	Choice
Language and data	Python 3.11+, pandas, numpy, standard-library `ipaddress` for IPv4 and IPv6
Machine learning	scikit-learn GradientBoostingClassifier, class-imbalance handling, time-based split
Reporting	matplotlib plots, JSON metrics, Markdown comparison reports, YAML configuration
Quality	pytest suite with 84 tests (ROV edge cases, time alignment, matching maths, bootstrap, explain schema, CLI round trip)
Interface	Command-line tool: `python -m src.cli explain | list-alerts`

7. Timeline of Delivery

Milestones are taken from the repository's commit history.

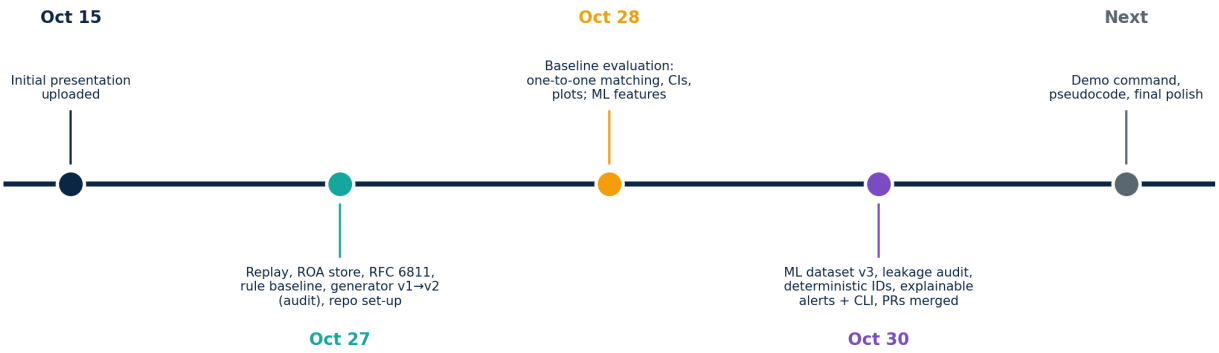


Figure 7. Delivery timeline.

Date (2026)	Milestone	Evidence in the repository
15 Oct	Initial presentation submitted	Presentation uploaded to the repository
27 Oct	Foundations and baseline	Replay pipeline, time-aligned ROA store, prefix inventory; RFC 6811 validation and rule-based baseline; INVENTORY_MISMATCH fix; v1 audit preserved; THIRD_PARTY_INVALID; generator v2 with forged-origin and split policy-lag scenarios; repository clean-up
28 Oct	Baseline evaluation and ML preparation	One-to-one event matching, event and alert precision, recall, latency, bootstrap CIs, multi-seed run and plots; past-only feature extraction and ML dataset configuration; first pull request merged
30 Oct	ML, audit and explainability	ML dataset v3 with leakage fixes; time-split GBDT training and comparison with the rules; deterministic alert IDs; dataset v3 changelog; evidence-based alerts and the explain / list-alerts CLI; second pull request merged
Remaining	Demonstration and polish	Demo command and mentor walkthrough with the pseudocode above; final consistency pass over the alert severity policy

8. References

Standards and specifications

- RFC 6811, *BGP Prefix Origin Validation*, https://www.rfc-editor.org/rfc/rfc6811.html (primary reference for the validator)
- RFC 6482, *A Profile for Route Origin Authorizations (ROAs)* (ROA prefix, origin ASN and maxLength semantics)
- RFC 4271, *A Border Gateway Protocol 4 (BGP-4)* (longest-prefix-match behaviour behind subprefix hijacks)

Data formats supported by the loaders

- RIPE RIS and RouteViews BGP update collectors (raw collector dump format)
- Historical RPKI ROA snapshot archives (prefix, ASN, maxLength per snapshot time)

Tools and libraries

Tool	Used for
Python, pandas, numpy	Replay, feature extraction, evaluation
scikit-learn	Gradient-boosted classifier, ROC / PR analysis, permutation controls
matplotlib	Evaluation and audit plots
PyYAML	Scenario and dataset configuration
pytest	84 automated tests
Git and GitHub	Version control, pull requests and mentor oversight
AI coding assistant and chat assistant	Implementation support and review of results; every reported number was reproduced by the project's own code

9. Conclusion

ThirdEye delivers the requested BGP replay pipeline, a ROA-aware anomaly detector with a rule-based baseline and one ML model, evidence-based alerts, and an evaluation of both anomalous and legitimate cases. The headline results are:

- The RFC 6811 rule baseline reaches event-level precision 0.6154, recall 0.8000 and F1 0.6957 on the baseline dataset, catching every exact-origin, subprefix and post-ROA-change hijack, with no false alarms on ordinary background traffic.
- It misses every forged-origin more-specific attack, which is exactly where origin-only checks are blind without AS_PATH analysis.
- One ML model recovers about 81% of those attacks and raises overall test recall from 0.72 to 0.92, with event F1 of 0.91 on the held-out test slice, at the cost of some false positives on traffic-engineering events and higher per-update latency.
- Every alert explains itself: covering ROAs, snapshot time, ROV state, inventory match, peer and a plain-English reason, plus score and top contributing features for ML alerts.

Why it matters

Operators need to know *why* an announcement was flagged and how often the detector is wrong. By measuring false positives on legitimate multi-origin and policy-change scenarios, and by publishing the audit trail of what was wrong in earlier versions, the project gives operators an honest picture of what ROA-aware monitoring can and cannot do, and where feed metadata starts to help.

Limitations

- All data is synthetic, so results show how separable the modelled scenarios are, not accuracy on real operator feeds, which will be noisier.
- Several benign scenarios are flagged by construction to expose ROA and inventory lag, so baseline precision reflects the scenario mix rather than a real-world false-positive rate.
- Replay-time alert latency is zero because alerts fire on arrival; wall-clock processing time is the meaningful latency figure.
- Forged-origin detection without AS_PATH analysis is inherently limited, and the ML residual separability and shuffled-control spread noted in section 5 remain.

Next steps

- **Demonstration:** run the mentor walkthrough using the pseudocode in section 6 and the explain / list-alerts commands.
- **Real data:** replay a real RIPE RIS or RouteViews window with matching historical ROA snapshots and compare against these synthetic results.
- **Severity policy:** finalise how announcements that precede the first ROA snapshot are rated, and map ML scores to severity bands.
- **Operator adoption:** keep the inventory current, review INVENTORY_MISMATCH and ROA_INVALID alerts as policy-change signals, and treat ML alerts as prompts for human review. The tool deliberately performs no automatic blocking.